ASSIGNMENT 15.2

K. Keerthana
2303A52027
Batch-38

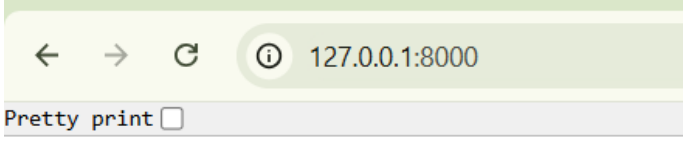
**Task Descriptor-1: (Setup Fast API Backend )**
Instructions:
• Install Fas tAPI and Uvicorn.
• Use AI to generate a basic FastAPI application with one endpoint
**Sample Code:**
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
return {"message": "Welcome to AI-powered FastAPI"}
**Expected Output:**
Running the server and accessing:
http://127.0.0.1:8000/
Response:
{"message": "Welcome to AI-powered FastAPI"}

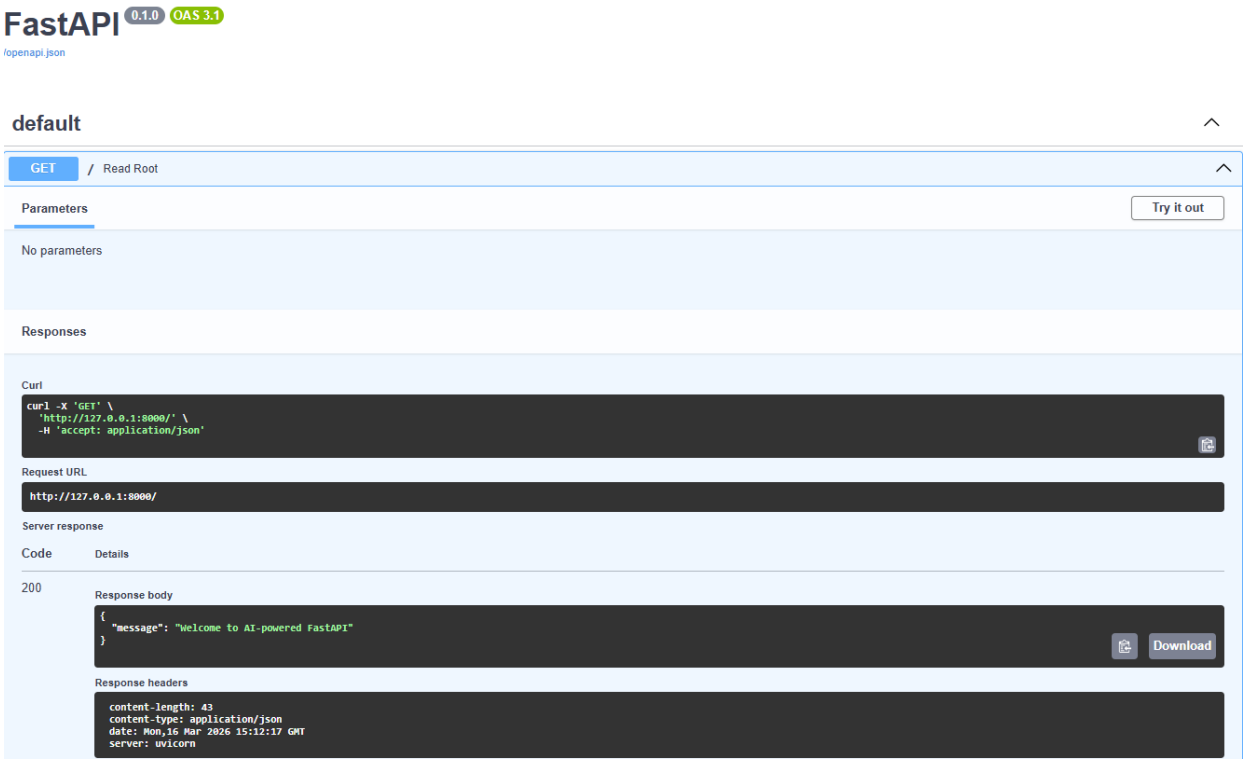
**Code:**

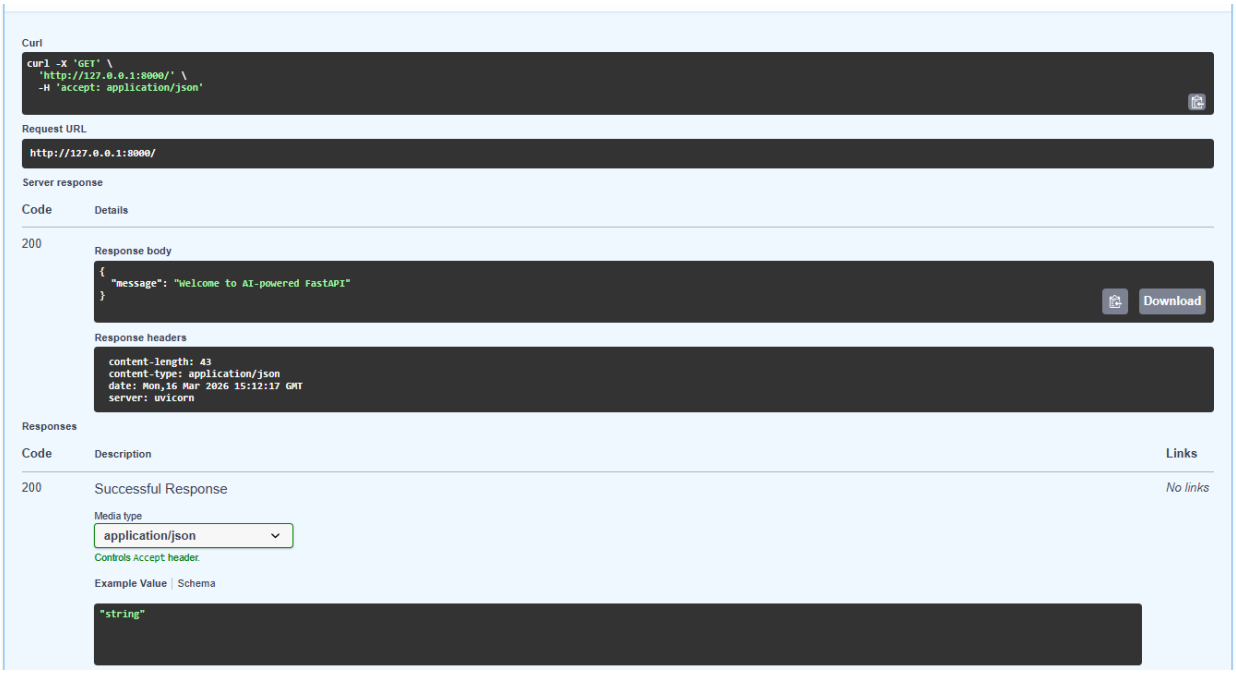
main.py > read_root

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
```

**Output:**






**Task Descriptor- 2: (API – Create and Read Items)**

Instructions:

• Implement endpoints to add and retrieve products.

• Use a Python list as temporary storage.

**Sample Code:**

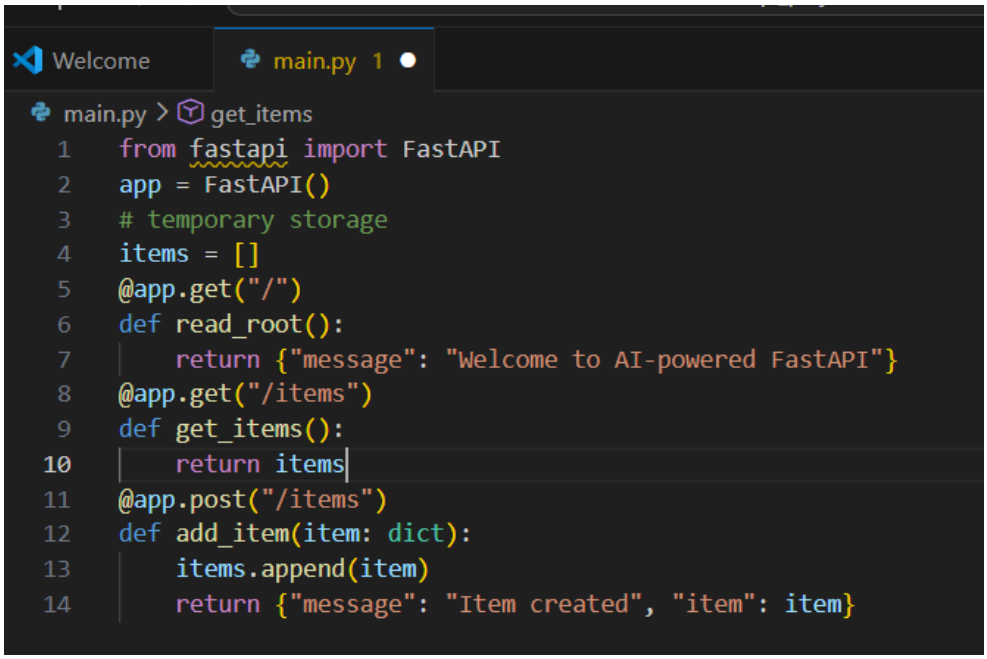
```
items = []
@app.get("/items")
def get_items():
```

return items
@app.post("/items")
def add_item(item: dict):
items.append(item)
return {"message": "Item created", "item": item}

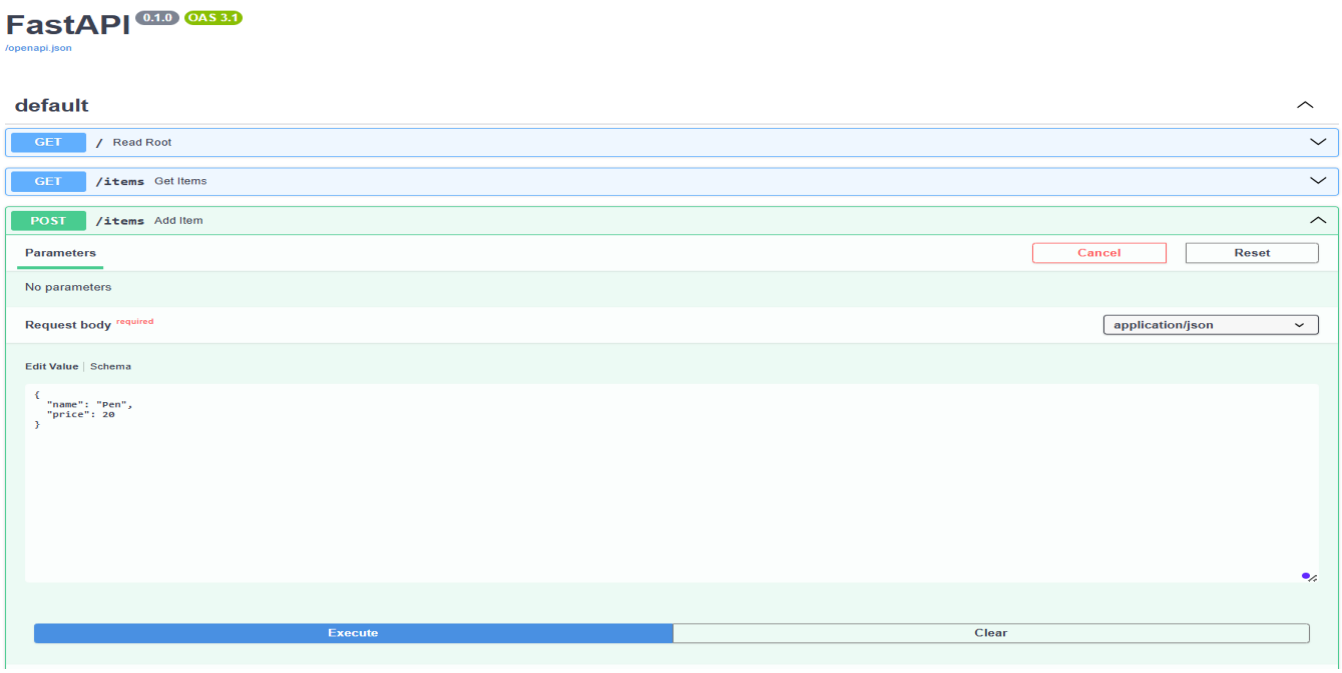
**Expected Output:**

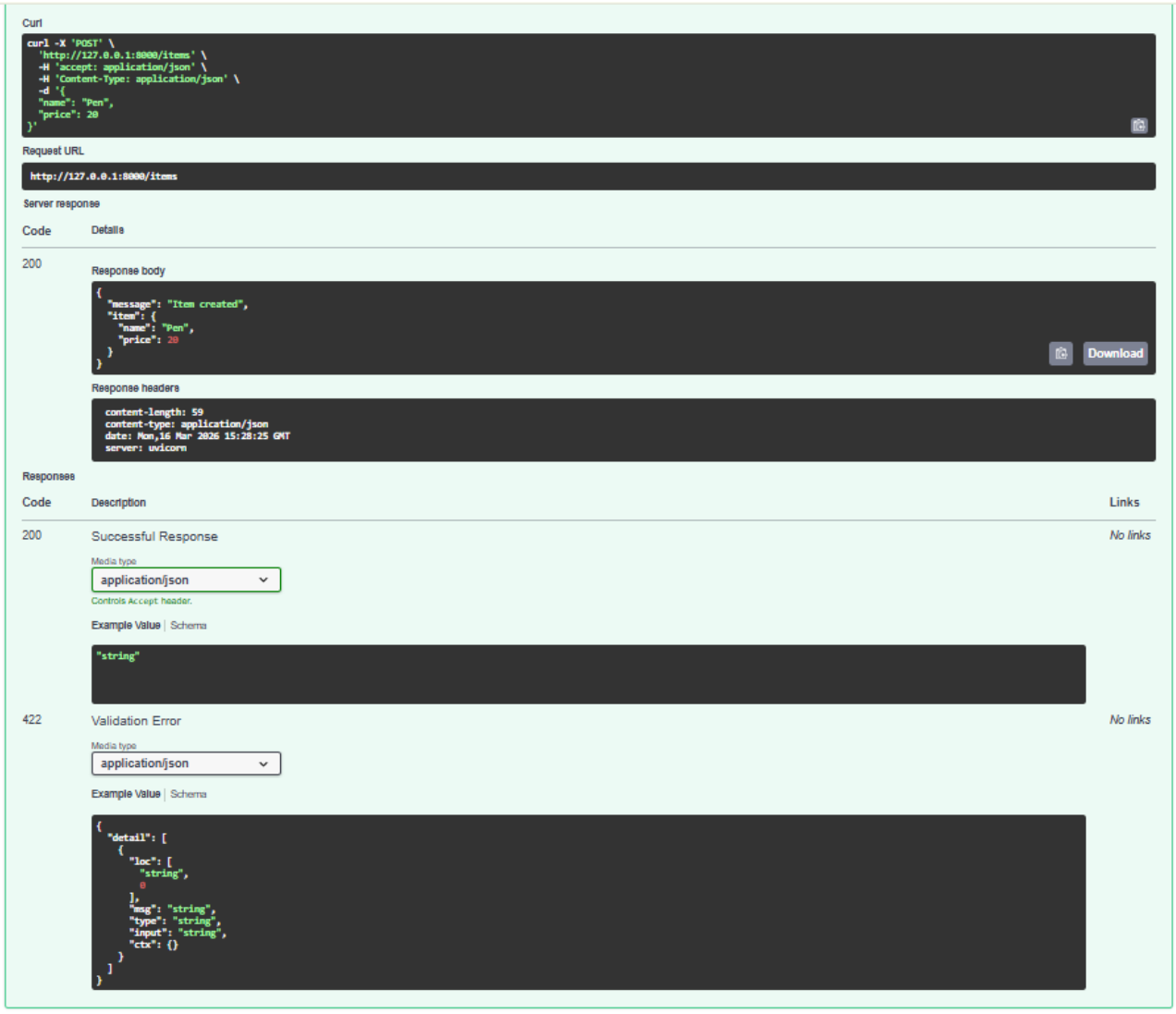
GET /items → []
POST /items with {"name":"Pen","price":20}
→ {"message":"Item created","item":{"name":"Pen","price":20}}

**Code:**




**Output:**






**Task Descriptor- 3: (Update Existing Item)**
Instructions:
• Use AI to generate an endpoint to update an item using its index.
**Sample Code:**
@app.put("/items/{index}")
def update_item(index: int, item: dict):
if index >= len(items):
return {"error": "Item not found"}
items[index] = item
return {"message": "Item updated", "item": item}
**Expected Output:**
PUT /items/0 with {"name":"Pencil","price":10}
→ {"message":"Item updated","item":{"name":"Pencil","price":10}}

**Code:**

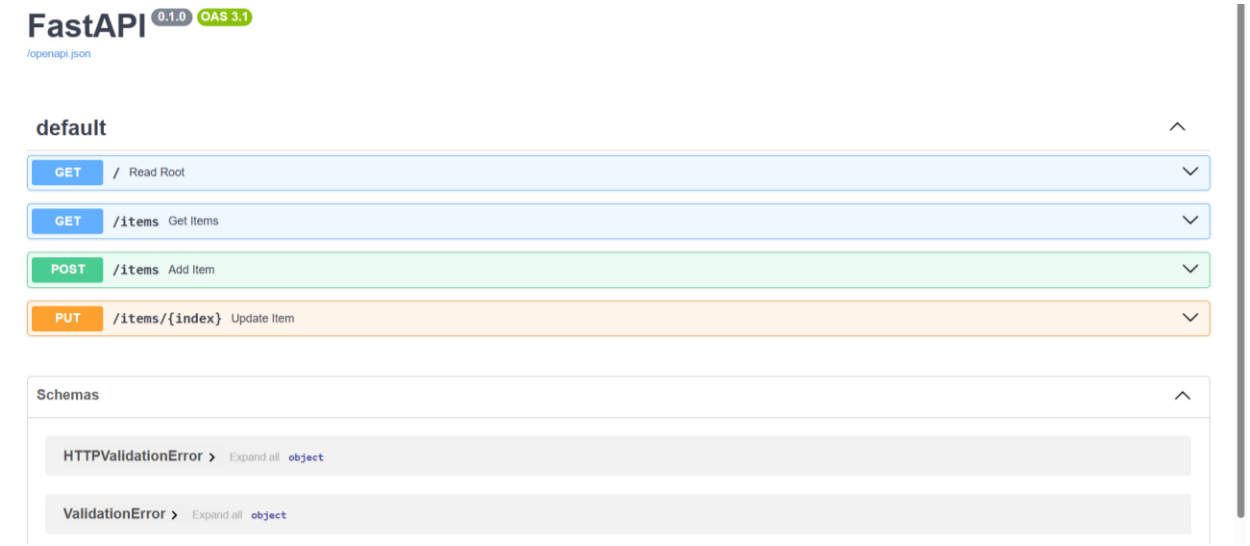
Welcome | main.py 1

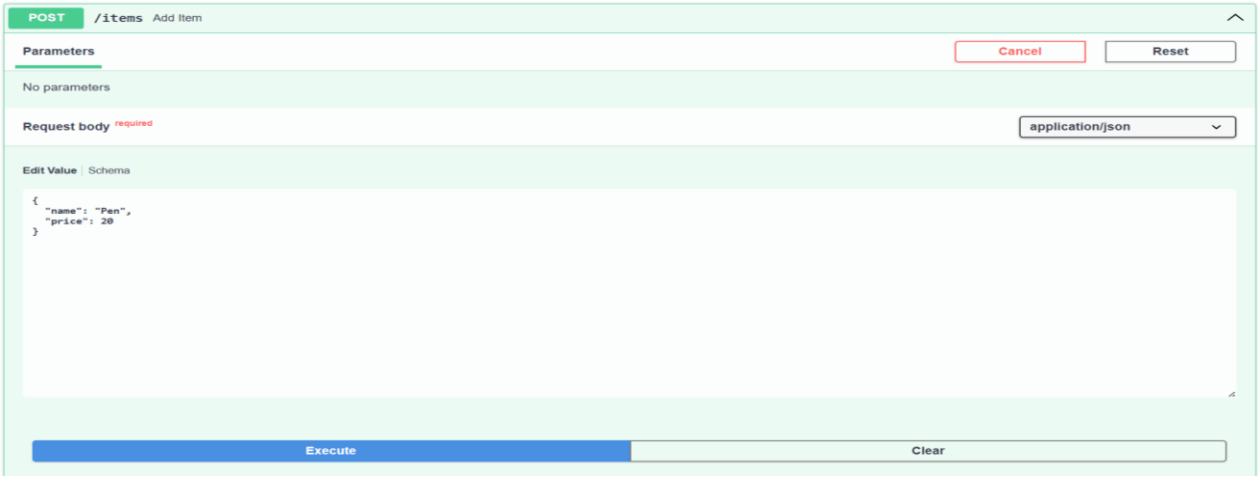
main.py > update_item

```python
from fastapi import FastAPI

app = FastAPI()

# temporary storage
items = []

# root endpoint
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}

# get all items
@app.get("/items")
def get_items():
    return items

# add item
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

```python
# add item
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}

# update item using index
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}

    items[index] = item
    return {"message": "Item updated", "item": item}
```

## Output:


FastAPI
0.1.0
OAS 3.1
/openapi.json
default
GET / Read Root
GET /items Get Items
POST /items Add Item
PUT /items/{index} Update Item
Schemas
HTTPValidationError
Expand all object
ValidationError
Expand all object


POST /items Add Item
Parameters
Cancel
Reset
No parameters
Request body required
application/json
Edit Value | Schema
{
"name": "Pen",
"price": 20
}
Execute
Clear


Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/items' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"name": "Pen",
"price": 20
}'
Request URL
http://127.0.0.1:8000/items
Server response
Code
Details
200
Response body
{
"message": "Item created",
"item": {
"name": "Pen",
"price": 20
}
}
Download
Response headers
content-length: 59
content-type: application/json
date: Mon,16 Mar 2026 15:37:53 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema

Responses

| Code | Description | Links |
|---|---|---|
| 200 | Successful Response | *No links* |

Media type: application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

| Code | Description | Links |
|---|---|---|
| 422 | Validation Error | *No links* |

Media type: application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

PUT /items/{index} Update Item

Parameters

Cancel | Reset

| Name | Description |
|---|---|
| index * required<br>integer<br>(path) | 0 |

Request body required — application/json

Edit Value | Schema

```
{
  "name": "Pencil",
  "price": 10
}
```

Execute | Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/items/0' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "name": "Pencil",
  "price": 10
}'
```

Request URL

```
http://127.0.0.1:8000/items/0
```

Server response

| Code | Details |
|---|---|
| 200 | Response body |

```
{
  "message": "Item updated",
  "item": {
    "name": "Pencil",
    "price": 10
  }
}
```

Download

Response headers

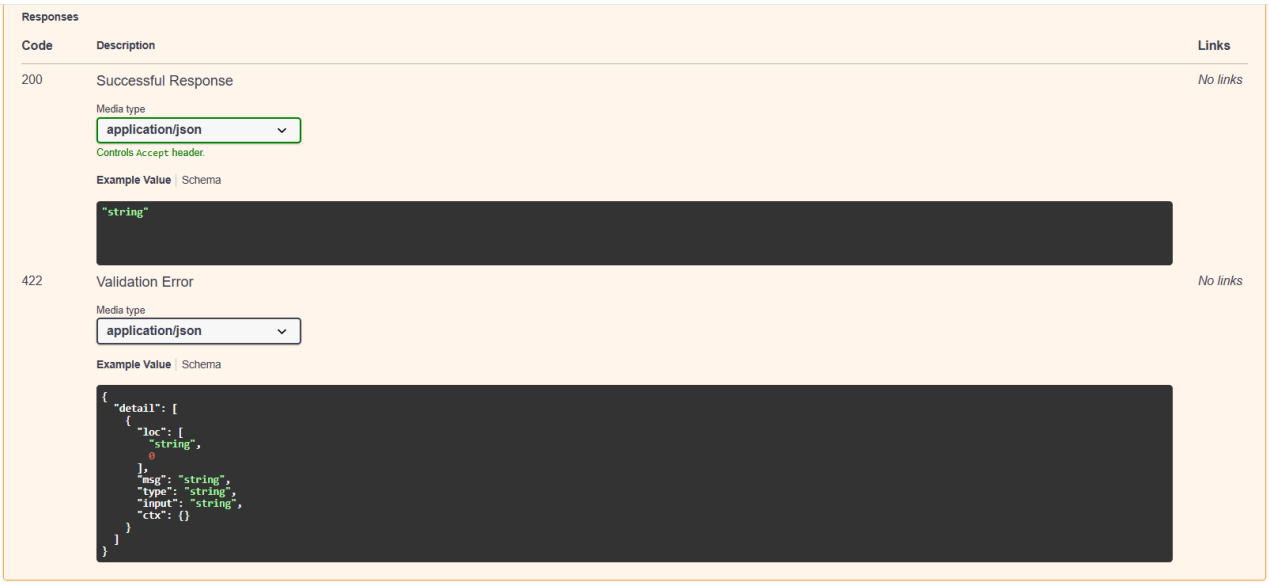
```
content-length: 62
content-type: application/json
date: Mon,16 Mar 2026 15:39:57 GMT
server: uvicorn
```

Responses

| Code | Description | Links |
|---|---|---|
| 200 | Successful Response | *No links* |

Media type: application/json

Controls Accept header.




**Task Descriptor- 4: (Delete Item)**

Instructions:

• Implement deletion of an item by index.

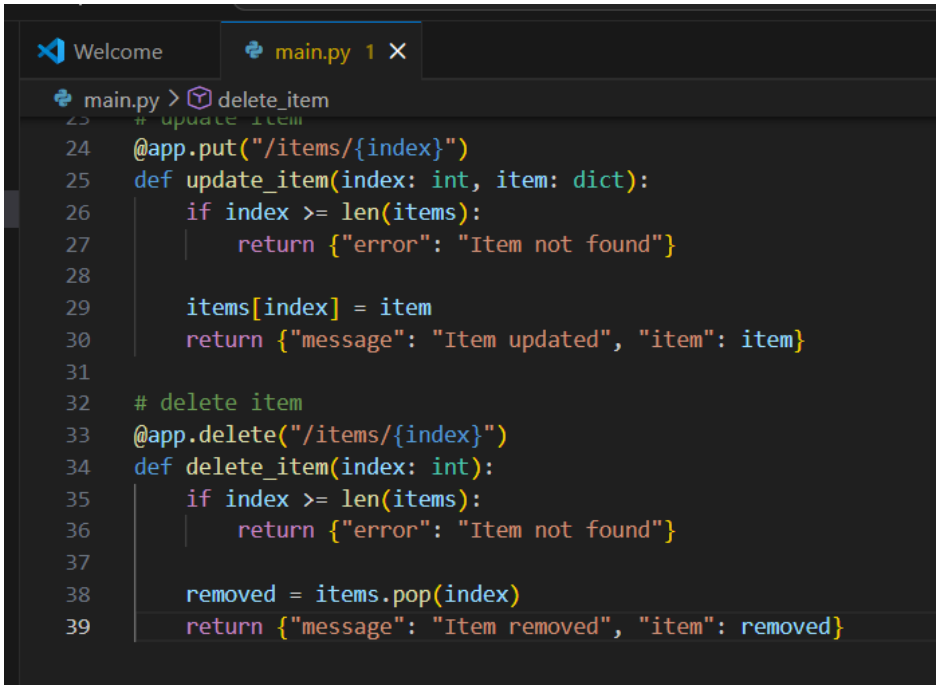
**Sample Code:**

@app.delete("/items/{index}")
def delete_item(index: int):
if index >= len(items):
return {"error": "Item not found"}
removed = items.pop(index)
return {"message": "Item removed", "item": removed}

**Expected Output:**

DELETE /items/0
→ {"message":"Item removed","item":{"name":"Pencil","price":10}}

**Code:**

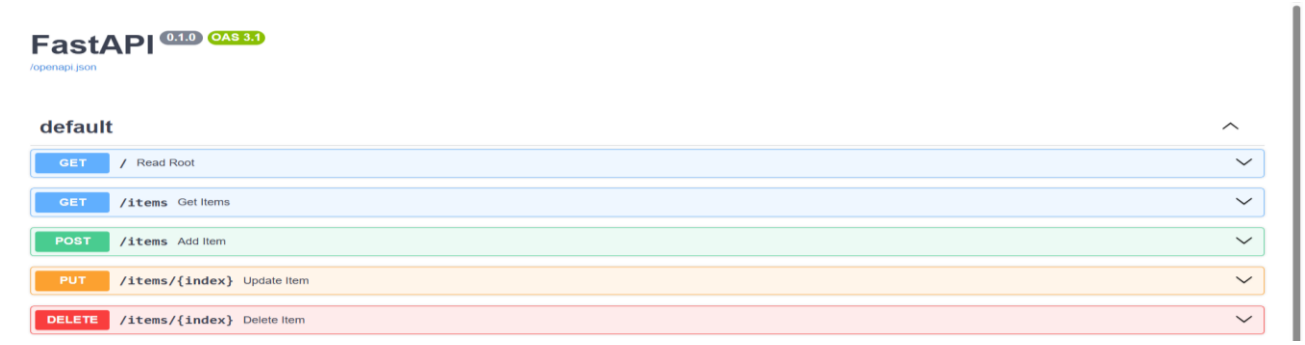


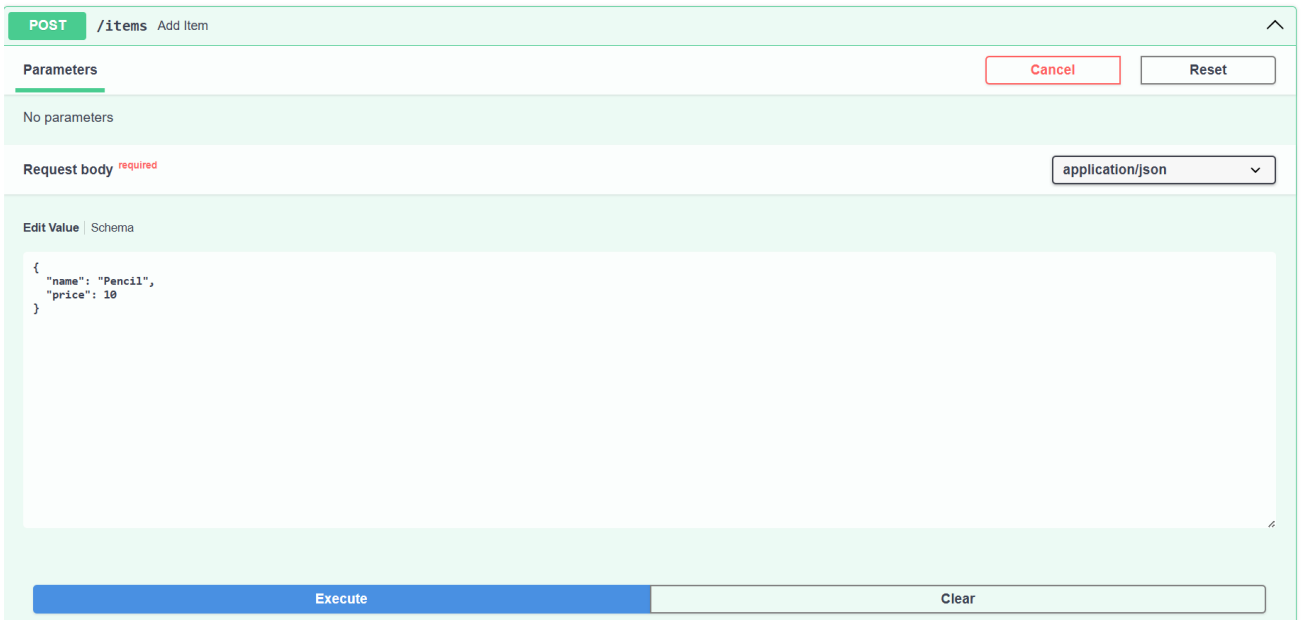
**Output:**

Welcome | main.py 1

main.py > delete_item

```python
# update item
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}

    items[index] = item
    return {"message": "Item updated", "item": item}

# delete item
@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items):
        return {"error": "Item not found"}

    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```



Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/items' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "name": "Pencil",
  "price": 10
}'
```

Request URL

```
http://127.0.0.1:8000/items
```

Server response

| Code | Details |
|---|---|
| 200 | Response body |

```
{
  "message": "Item created",
  "item": {
    "name": "Pencil",
    "price": 10
  }
}
```

Download

Response headers

```
content-length: 62
content-type: application/json
date: Mon,16 Mar 2026 15:48:22 GMT
server: uvicorn
```

Responses

| Code | Description | Links |
|---|---|---|
| 200 | Successful Response | *No links* |

Media type: application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

| | | |
|---|---|---|
| 422 | Validation Error | *No links* |

Media type: application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

DELETE /items/{index} Delete Item

Parameters

Cancel

| Name | Description |
|---|---|
| index * required<br>integer<br>*(path)* | 0 |

Execute | Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/items/0' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/items/0
```

Server response

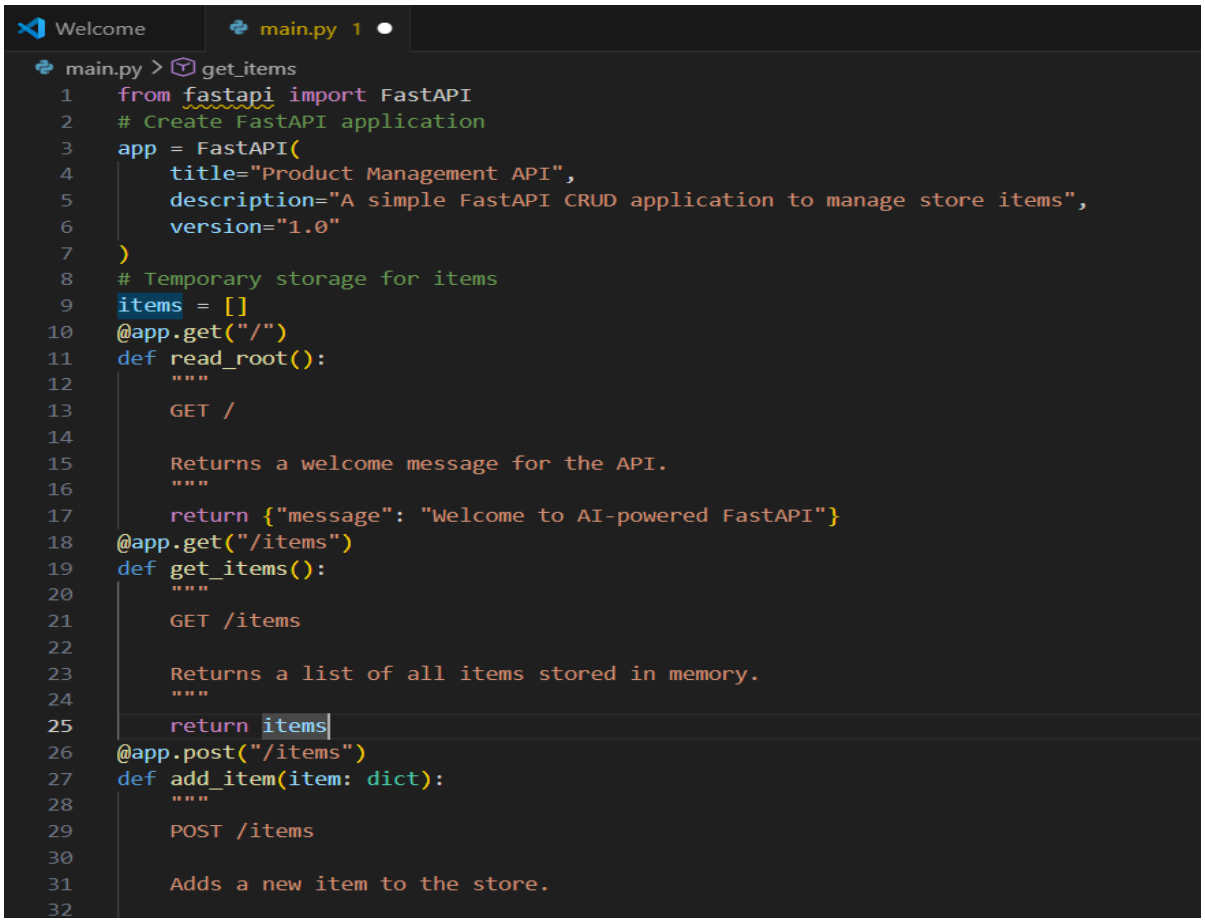
| Code | Details |
|---|---|



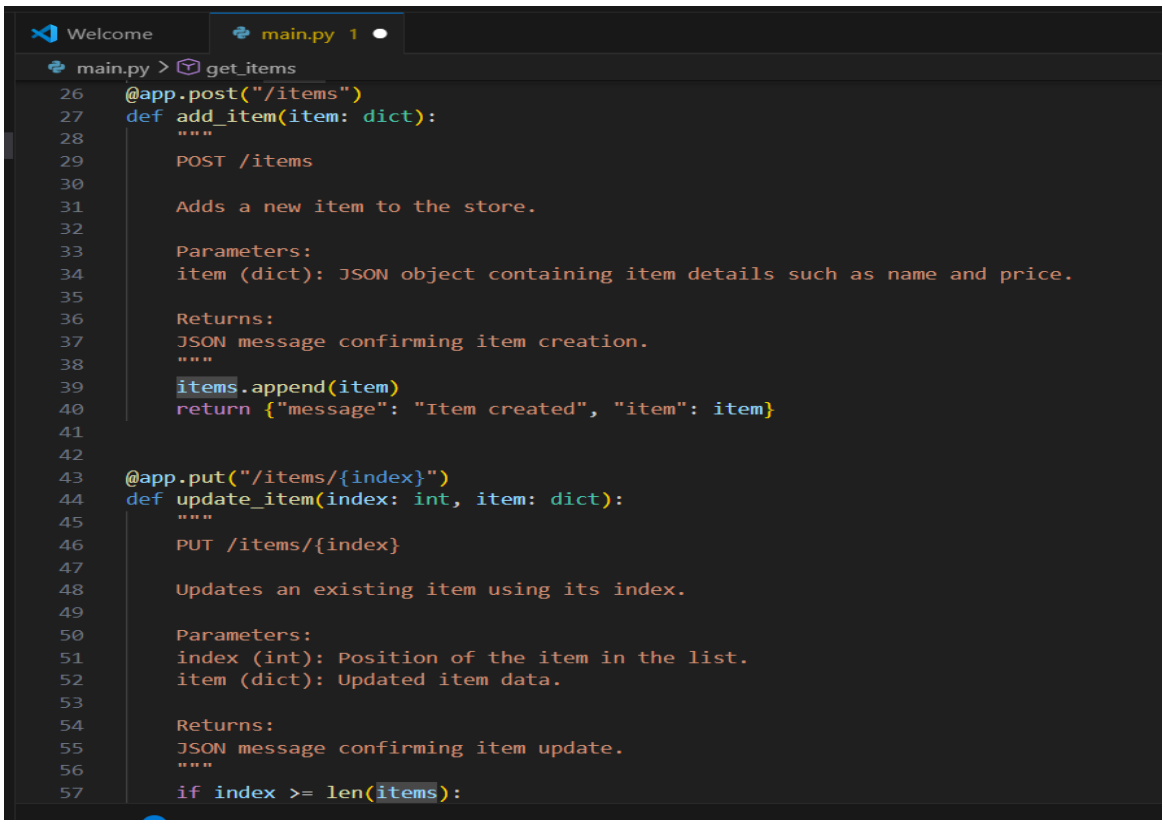
**Task Descriptor -5: (Add Auto-Generated Documentation)**
Instructions:
• Use AI to add inline comments and docstrings for all endpoints.
• Optionally, integrate Swagger / Flask-RESTX to auto-generate API documentation.
**Sample Code:**

```
@app.route('/items', methods=['GET'])
def get_items():
"""
GET /items
Returns a list of all items in the store.
```

**Code:**



```python
from fastapi import FastAPI
# Create FastAPI application
app = FastAPI(
    title="Product Management API",
    description="A simple FastAPI CRUD application to manage store items",
    version="1.0"
)
# Temporary storage for items
items = []
@app.get("/")
def read_root():
    """
    GET /

    Returns a welcome message for the API.
    """
    return {"message": "Welcome to AI-powered FastAPI"}
@app.get("/items")
def get_items():
    """
    GET /items

    Returns a list of all items stored in memory.
    """
    return items
@app.post("/items")
def add_item(item: dict):
    """
    POST /items

    Adds a new item to the store.

```



```python
@app.post("/items")
def add_item(item: dict):
    """
    POST /items

    Adds a new item to the store.

    Parameters:
    item (dict): JSON object containing item details such as name and price.

    Returns:
    JSON message confirming item creation.
    """
    items.append(item)
    return {"message": "Item created", "item": item}


@app.put("/items/{index}")
def update_item(index: int, item: dict):
    """
    PUT /items/{index}

    Updates an existing item using its index.

    Parameters:
    index (int): Position of the item in the list.
    item (dict): Updated item data.

    Returns:
    JSON message confirming item update.
    """
    if index >= len(items):
```

```python
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}

    items[index] = item
    return {"message": "Item updated", "item": item}


@app.delete("/items/{index}")
def delete_item(index: int):
    """
    DELETE /items/{index}

    Deletes an item from the list using its index.

    Parameters:
    index (int): Position of the item to remove.

    Returns:
    JSON message confirming deletion.
    """
    if index >= len(items):
        return {"error": "Item not found"}

    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

**Output:**